

Executive Summary

NIGHTFRAME asks a systems question: **How can independently owned, heterogeneous devices cooperate as an adaptive compute and connectivity platform without assuming uniform hardware, continuous infrastructure, or unbounded operator consent?**

The software answers through five interacting abstractions:

1. **Capability manifests** convert device heterogeneity into schedulable facts.
2. **Role assignment and shard coordination** convert those facts into work placement.
3. **Mesh and cellular intelligence** treat connectivity and location as observable, changing resources.
4. **Probation, identity, and contribution accounting** provide a preliminary trust-and-incentive structure.
5. **Consent and resource limits** represent the human operator as a governing participant rather than an invisible source of resources.

The artifact contains executable implementations of significant portions of this design, including gRPC contracts, node registration, heartbeat processing, role assignment, inference-shard scheduling, ONNX execution, local discovery, RF fingerprinting, handover prediction, a persistent ledger, resource limits, and simulation tools. It also contains explicit placeholders for result signature verification, production consensus, secure propagation, model hydration, and complete prompt-result processing.

Accordingly, this paper presents NIGHTFRAME as a **research platform and conceptual systems framework**. It does not claim validated production security, planetary scale, guaranteed RF accuracy, or complete distributed inference.

Research Framing

Problem Statement

Edge environments contain useful but fragmented resources: CPU time, GPU capacity, storage, network reachability, radio observations, and human-granted permission to share them. These resources vary over time and are distributed across devices with unequal capabilities and reliability.

A cooperative edge platform must therefore solve a joint problem:

$$\max_{\pi} U_{\text{compute}} + U_{\text{connectivity}} + U_{\text{knowledge}} - C_{\text{resource}} - C_{\text{risk}}$$

subject to:

$$r_i(t) \leq \bar{r}_i, \quad a_i(t) \in A_i, \quad q_j(t) \geq q_j^{\min}, \quad \tau_i(t) \geq \tau^{\min}, \quad c_i = 1.$$

Here, π is a scheduling and coordination policy, $r_i(t)$ is resource use at node i , $\bar{r}_i$ is its configured limit, $a_i(t)$ is its assigned role, $q_j(t)$ is task quality, $\tau_i(t)$ is trust, and c_i is recorded operator consent.

Research Questions

The artifact motivates four research questions:

- **RQ1:** Can capability declarations and live telemetry support useful role assignment across heterogeneous devices?
- **RQ2:** Can a thin coordination plane organize distributed inference while keeping heavy data and computation at the edge?
- **RQ3:** Can network, cellular, and resource observations jointly improve continuity and placement decisions?
- **RQ4:** Can trust, incentives, consent, and hard limits be modeled as first-class technical constraints rather than external policy concerns?

Contributions

NIGHTFRAME contributes an integrated research artifact with:

- a capability-aware node model spanning compute, storage, networking, and cellular observations;
- a thin-orchestrator architecture using REST, SignalR, and gRPC;
- a layer-range scheduling model for pipeline-oriented inference;
- a combined RF localization and predictive handover subsystem;
- a preliminary trust lifecycle based on cryptographic identity and probation;
- a contribution ledger connected to work completion;
- explicit operator consent and resource-cap models;
- simulation and self-test facilities for selected networking behaviors; and
- a clear boundary between implemented, simulated, and proposed mechanisms.

Method

This paper uses **artifact-centered conceptual analysis**. No outside sources, benchmarks, or datasets are used. Claims are derived from source code, configuration, protocol definitions, documentation, and executable test or simulation paths in this repository.

The analysis proceeded through:

1. repository inventory and build/configuration inspection;
2. tracing control flow from Orchestrator and Drone entry points;
3. examining shared interfaces and the gRPC protocol;
4. classifying mechanisms by implementation maturity;
5. extracting recurring design concepts;
6. expressing those concepts as formal models and testable hypotheses; and
7. identifying falsification criteria and residual risks.

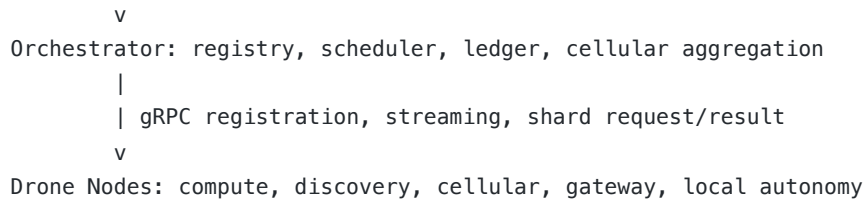
This method supports architectural interpretation, not empirical proof. Comments, type names, or unexecuted branches are treated as design intent unless connected to an executable path.

Platform Architecture

System Decomposition

NIGHTFRAME separates coordination from execution:

```
Administrative Console
|
| REST + SignalR
```



The **Orchestrator** stores metadata, admits nodes, assigns roles, creates shard plans, tracks heartbeats, aggregates cellular observations, and records ledger entries. The **Drone** audits its hardware, creates or loads an identity, discovers peers, optionally observes cellular conditions, requests work, and executes ONNX or fallback computation. Consoles observe and control selected state through REST and SignalR.

Protocol Model

The protocol expresses each node as a manifest:

$$M_i = \langle I_i, V_i, H_i, K_i, P_i, C_i, N_i, L_i \rangle$$

where:

- I_i is node identity;
- V_i is binary version;
- H_i is hardware capacity;
- K_i is cached model state;
- P_i is the public key;
- C_i is cellular capability;
- N_i is neural-compute capability; and
- L_i is location information.

This is a conceptual shift from addressing a device by location alone to admitting it by an evolving bundle of capabilities.

Thin Coordination Plane

The Orchestrator is designed as a switchboard rather than the primary compute engine. Let total work be W , Orchestrator work be W_o , and edge work be W_e . The intended architecture seeks:

$$W = W_o + W_e, \quad W_o \ll W_e$$

while the Orchestrator retains enough state to make placement and trust decisions. This reduces central compute pressure but creates dependencies on accurate manifests, reliable streams, secure result verification, and available model shards.

Conceptual Frameworks

Framework 1: Capability-to-Role Transformation

The registry transforms device measurements into operational roles. In the current implementation, GPU presence, RAM, and free disk determine whether a node becomes compute, storage, general, or relay.

Formally:

$$\rho_i = f(H_i, K_i, S_i, T_i)$$

where ρ_i is role, H_i is hardware, K_i is model availability, S_i is live state, and T_i is trust state. The current function is threshold- and rule-based; the broader framework permits learned or optimization-based assignment.

This framework achieves two conceptual goals:

- **hardware agnosticism:** unlike devices can participate without pretending to be equivalent; and
- **graceful specialization:** low-capacity devices may relay or scout rather than being rejected solely for lacking compute power.

The current artifact does not yet continuously recompute roles from changing conditions, so dynamic reassignment remains a research direction.

Framework 2: Pipeline-Oriented Cooperative Inference

The shard coordinator models a neural model as an ordered layer interval:

$$\mathcal{L} = \{0, 1, \dots, L-1\}.$$

Given shard width k , the job is partitioned into:

$$S_m = [mk, \min((m+1)k-1, L-1)].$$

Each shard is assigned to a node and consumes the previous shard's output. For n shards, end-to-end latency is approximately:

$$T_j = T_{\text{load}} + T_{\text{infer}} + T_{\text{transfer}} + T_{\text{coord}}$$

The implementation creates the ordered assignments, waits for predecessor output, records completion, and credits work. The Drone can load an ONNX shard or execute a fallback transform. However, the repository does not include a validated model-partitioning pipeline that produces compatible layer-range ONNX artifacts. Thus, the scheduler is implemented as an architectural prototype while complete distributed model inference remains unproven.

Framework 3: Connectivity as an Intelligent Resource

NIGHTFRAME treats connectivity as more than a transport prerequisite. Local discovery, gateway state, cellular measurements, RF location, and handover prediction become inputs to system decisions.

An individual radio observation is represented conceptually as:

$$x_t = \langle \text{cell ID}, \text{RSRP}, \text{RSRQ}, \text{SINR}, \text{neighbors}, \text{timing}, \text{velocity} \rangle_t.$$

The RF locator stores labeled fingerprints and predicts location through distance-weighted nearest neighbors:

$$\hat{p}(x) = \text{weighted_location_average}(N_K(x))$$

with:

$$w_j = 1 / (d(x, x_j) + \epsilon)$$

The handover subsystem observes time-series signal change. Its rule-based fallback estimates signal fade using a regression slope and recommends a neighbor when the serving signal crosses a threshold, degrades rapidly, or is dominated beyond hysteresis. An ONNX sequence-model path is also represented.

The framework's larger implication is that **network state can become part of compute placement and continuity planning**. That integration is conceptually present but not yet closed-loop in the scheduler.

Framework 4: Trust as a Lifecycle

NIGHTFRAME models trust as a process rather than a registration-time Boolean:

$\text{unknown} \rightarrow \text{admitted} \rightarrow \text{probationary} \rightarrow \text{active}$.

Nodes create a key pair and derive an identifier from the public key hash. Registration includes the public key, and new nodes begin in probation. Shadow test results can promote or remove them.

A mature trust score could be represented as:

$$\tau_i(t+1) = \alpha \tau_i(t) + \beta v_i(t) + \gamma u_i(t) - \delta e_i(t),$$

where v_i is verified work, u_i is uptime, and e_i is detected error or fraud. The current artifact implements identity creation, local signing, probation counters, and penalty accounting, but compute-result signatures are not verified by the Orchestrator and production consensus is explicitly unfinished. Trust is therefore an implemented lifecycle skeleton, not a complete adversarial defense.

Framework 5: Contribution Accounting

The ledger maps activity to credits and debits:

$$B_i(t) = \sum_{\ell \in E_i(t)} a_{\ell},$$

where $E_i(t)$ is the set of entries associated with node i and a_{ℓ} is a signed amount. Compute, storage, relay, uptime, prompts, and penalties are represented as transaction categories.

This creates a conceptual feedback loop:

$\text{contribution} \rightarrow \text{record} \rightarrow \text{balance} \rightarrow \text{future access or priority}$.

The persistent ledger and compute crediting path are implemented. Enforcement is partial: prompt debit logic exists, but the public prompt endpoint does not bind submissions to an authenticated node account. The ledger should therefore be interpreted as an internal accounting experiment rather than a financial system.

Framework 6: Operator-Governed Autonomy

The platform's resource model includes explicit caps for bandwidth, guests, CPU, RAM, GPU, storage, and log retention. Gateway setup includes a five-part consent flow covering bandwidth, billing, service terms, IP-address liability, and limits.

For each resource r , admissible autonomous action requires:

$$r_{\text{requested}} \leq r_{\text{available}} \leq r_{\text{operator cap}}.$$

The conceptual contribution is important: autonomy is bounded by the device owner's declared constraints. Yet declarations alone do not prove enforcement. Several limits are constants or configuration values that require further integration and adversarial testing.

Framework 7: Layered Resilience

The artifact combines:

- heartbeat-based liveness;
- reconnect attempts with exponential backoff;
- pending-job queues;
- local peer discovery;
- simulated relay paths;
- gossip-style update descriptions;
- process supervision; and
- local/offline behavior.

This suggests a layered resilience model:

$$R_{\text{system}} = g(R_{\text{process}}, R_{\text{node}}, R_{\text{link}}, R_{\text{task}}, R_{\text{update}}).$$

Failure in one layer should not imply complete system failure. Some layers are executable, while secure gossip transport, verified updates, and robust distributed recovery remain incomplete.

Mechanism Maturity

Mechanism	Artifact evidence	Classification
Node manifest and role assignment	gRPC protocol and DroneRegistry	Implemented prototype
Heartbeat/liveness tracking	streaming service and monitor	Implemented prototype
Layer-range shard scheduling	ShardCoordinator	Implemented prototype
ONNX local shard execution	ComputeEngine	Implemented prototype
Model partition production and hydration	placeholders and download path	Partial/proposed
Contribution ledger	LiteDB-backed LedgerService	Implemented prototype
RF fingerprint localization	KNN model and training path	Implemented prototype
Handover prediction	rule-based path and ONNX path	Implemented prototype
UDP/mDNS discovery	MeshDiscovery	Implemented prototype
Gossip update propagation	in-memory selection/logging	Simulated/proposed transport
Result signature verification	explicit TODO	Not implemented
Production consensus	explicit TODO	Not implemented
Operator consent flow	setup workflow	Implemented prototype
Hard resource-limit enforcement	constants and selected integrations	Partial

Network scale behavior	simulator	Simulated only
------------------------	-----------	----------------

Security, Safety, Ethics, and Governance

Threat Model

Relevant threats include:

- malicious nodes submitting fabricated results;
- identity-file theft or substitution;
- unauthorized Orchestrator access;
- untrusted update or model payloads;
- replayed or forged telemetry;
- privacy leakage from location and cellular observations;
- abuse of gateway traffic;
- resource exhaustion;
- unsafe privileged networking operations; and
- misleading confidence in simulated or placeholder mechanisms.

Current Controls

The artifact includes local cryptographic identity, signing capability, probation, heartbeat monitoring, consent records, resource-limit definitions, session logging, version checks, and hash fields for updates and results.

Material Gaps

The current system lacks a complete production authentication and authorization model. Result signature verification is unfinished. Gossip verification accepts structurally plausible node identifiers rather than validating signatures against trusted public keys. Consensus is unfinished. Gateway operation carries legal, privacy, and security implications that consent text alone cannot resolve.

The ethical operating principle for the artifact is therefore:

No feature that consumes another person's compute, bandwidth, location data, or network identity should be treated as authorized merely because the code can attempt it.

Evaluation Framework

Hypotheses

- **H1:** Capability-aware assignment reduces task failures relative to uniform random assignment.
- **H2:** Layer-range scheduling improves feasible model size across constrained nodes, while network transfer creates a measurable latency tradeoff.
- **H3:** RF and neighbor-cell histories improve continuity decisions relative to reactive threshold-only handover.
- **H4:** Probation and redundant verification reduce acceptance of incorrect results, provided signatures and consensus are fully implemented.

- **H5:** Consent and enforced caps reduce operator resource violations without eliminating useful contribution.

Metrics

Dimension	Metrics
Compute	completion rate, latency, throughput, model load time, transfer overhead
Placement	role accuracy, reassignment count, capacity utilization, rejected jobs
Resilience	recovery time, queued-job age, partition tolerance, update reach
Cellular	localization error, handover precision/recall, interruption duration
Trust	false acceptance, false rejection, probation duration, verified-work ratio
Governance	cap violations, consent revocation latency, resource-use accuracy
Reproducibility	clean-clone build rate, deterministic simulation results, documented prerequisites

Experimental Design

A defensible evaluation should progress through four stages:

1. **Clean-clone verification:** build portable core and web console on fresh runners using lockfiles and documented prerequisites.
2. **Deterministic simulation:** seed network simulations and compare scheduling, relay, and failure policies under controlled topology changes.
3. **Local multi-process integration:** run one Orchestrator and multiple isolated Drone instances with generated manifests and controlled failures.
4. **Permissioned hardware pilot:** evaluate compute, cellular, and gateway behavior only on owned devices and authorized networks.

Every result should report configuration, hardware, software version, seed, sample size, failure conditions, and raw measurements. Claims unsupported by those records should remain hypotheses.

Falsification Criteria

The architecture should be reconsidered if:

- coordination overhead dominates edge work under realistic node counts;
- pipeline transfer costs erase feasible compute benefits;
- manifests are too stale or inaccurate for useful role assignment;
- trust mechanisms cannot distinguish faulty from honest nodes at acceptable rates;
- RF predictions fail to improve continuity;
- resource caps cannot be reliably enforced; or
- clean-clone reproduction requires undocumented local state.

Reproducibility

Artifact Build

Portable core:

```
dotnet restore NIGHTFRAME.Core.sln
dotnet build NIGHTFRAME.Core.sln --no-restore --configuration Release
```

Web console:

```
cd gamma1-web
npm ci
npm run build
```

Selected Drone self-tests and simulation commands, after a successful build:

```
dotnet run --project Drone -- --self-test
dotnet run --project Drone -- --simulate --edge-cases
dotnet run --project Drone -- --simulate --all
```

These commands exercise prototype mechanisms; they are not substitutes for security audits, hardware validation, or real distributed inference tests.

Paper Export

PDF:

```
pandoc docs/WHITEPAPER.md \
  --from markdown+tex_math_dollars \
  --standalone \
  -o NIGHTFRAME-whitepaper.pdf
```

DOCX:

```
pandoc docs/WHITEPAPER.md --standalone -o NIGHTFRAME-whitepaper.docx
```

Limitations

This work has five principal limitations.

First, it is an artifact-derived white paper without external comparison. Second, several central security and consensus mechanisms are incomplete. Third, simulation classes cannot establish real-network scale or reliability. Fourth, privileged networking behavior is platform-, policy-, and hardware-dependent. Fifth, the repository does not contain validated model shards and benchmark evidence sufficient to prove useful end-to-end distributed inference.

These limitations bound the claims but do not erase the artifact's conceptual value. NIGHTFRAME integrates concerns often separated in prototypes: compute, radio state, trust, incentives, and operator governance.

Research Agenda

The highest-value next work is:

1. implement authenticated node sessions and verify every submitted result;
2. define deterministic consensus and adversarial test suites;
3. create a reproducible model-partitioning and hydration pipeline;
4. connect live network/cellular state to scheduler decisions;
5. convert resource-limit declarations into measured enforcement;
6. isolate privileged gateway behavior behind explicit platform adapters;
7. add deterministic integration tests and benchmark manifests; and
8. publish empirical results with raw data and exact artifact versions.

Conclusion

NIGHTFRAME provides a coherent research architecture for cooperative edge intelligence. Its strongest idea is not any single algorithm. It is the unification of capability-aware scheduling, distributed execution, connectivity intelligence, lifecycle trust, contribution accounting, and operator-governed autonomy.

The repository demonstrates that these concepts can share one protocol and one software platform. It also makes visible the hard remaining work: verified results, secure updates, complete model distribution, real-world evaluation, and enforceable governance. As a white-paper artifact, NIGHTFRAME establishes a testable systems framework and a concrete agenda for advancing it.

Declarations

Author Contributions

Malachi Hooper conceived and implemented the NIGHTFRAME software platform and is the author of this white paper.

Funding

No funding information is declared in the repository.

Conflicts of Interest

No conflicts of interest are declared.

Data and Code Availability

This paper uses only the NIGHTFRAME repository artifact. No external datasets or sources were used. Availability and reuse are governed by the repository's license and release settings.

Ethics Statement

Testing of gateway, cellular, discovery, propagation, or resource-sharing features must be limited to owned or explicitly authorized devices, accounts, radio hardware, and networks. Operator consent and applicable policy remain mandatory.

Artifact References

The following repository artifacts are the sole references for this paper:

- Orchestrator/Protos/nightframe.proto
- Orchestrator/Services/DroneRegistry.cs
- Orchestrator/Services/ShardCoordinator.cs
- Orchestrator/Services/LedgerService.cs
- Orchestrator/Services/CellularCoordinator.cs
- Orchestrator/Services/OrchestratorService.cs
- Orchestrator/Services/BackgroundServices.cs
- Drone/Core/DroneCore.cs
- Drone/Core/NodeIdentity.cs
- Drone/Compute/ComputeEngine.cs
- Drone/Cellular/RFLocatorModel.cs
- Drone/Cellular/HandoverPredictor.cs
- Drone/Network/MeshDiscovery.cs
- Drone/Network/GossipProtocol.cs
- Drone/Network/ResourceLimits.cs
- Drone/Setup/ConsentFlow.cs
- Drone/Testing/DroneTestHarness.cs
- Drone/Testing/NetworkSimulator.cs
- Shared/Interfaces/INetworkNode.cs
- Shared/Interfaces/INeuralCompute.cs